Bitcoin Commons: A Cryptographic Governance System for Bitcoin Development

BTCDecoded

2026

Abstract

Bitcoin faces a critical governance asymmetry: while its technical consensus layer is cryptographically bulletproof, its development governance relies on informal social coordination. At Bitcoin's multi-trillion dollar scale, this represents an existential vulnerability.

This whitepaper presents two innovations that enable each other: **BLVM** provides mathematical rigor (proofs locked to code, formal verification, consensus matching); **Bitcoin Commons** provides governance coordination without civil war (Ostrom's principles through cryptographic enforcement). Together they solve Bitcoin's governance asymmetry.

The work is organized around six core engineering tracks in the public BTCDecoded organization (spec, consensus, protocol, node, SDK, and governance), plus supporting repositories for packaging,

blvm-spec-lock verification, and the **blvm-commons** enforcement app. Engineering detail lives in technical documentation.

1. Introduction

Bitcoin solved Byzantine consensus between strangers (Nakamoto, 2008) but ignored consensus between developers. The network's substantial market capitalization demands institutional maturity matching its technical excellence.

The original cypherpunk developers focused on eliminating trusted third parties in transactions but inadvertently created trusted parties in development. Bitcoin Commons addresses Bitcoin's most critical vulnerability: governance asymmetry between technical consensus and development coordination.

1.1 The Talent Bottleneck: Orders of Magnitude and Sources

Bitcoin development draws on multiple hard domains simultaneously (C++, applied cryptography, distributed systems, security engineering, economics/game theory, and open-source governance). Each extra domain narrows the pool. Using conservative, sourced baselines and clearly labeled assumptions, the analysis below estimates the rarity of a contributor who combines these competencies and is available to work on Bitcoin:

Assumptions and sources:

- World population baseline: ~8.1B (UN DESA, World Population Prospects, 2022 Rev.)
- Global developers: ~30M-47M (range spanning widely cited industry estimates, incl. SlashData and similar studies)
- C++ share of developers: ~15%-25% (range spanning major annual developer surveys)
- Adult numeracy (problem-solving proficiency): on the order of 10%-20% globally (OECD PIAAC cross-country evidence; global extrapolation is approximate)
- Bitcoin Core maintainers: single-digit individuals; contributors: hundreds (public repo statistics)

Rarity funnel (indicative, overlapping, not strictly independent):

- Strong college-level math (calculus/linear algebra): 3%-5% of population, resulting in 240M-400M
- Professional developers: ~30M-47M (subset, separate baseline)
- C++/systems competency: 15%-25% of developers → 4.5M-11.8M
- Applied cryptography + Bitcoin protocol literacy: 1%-2% of C++ devs → 45k-236k
- Distributed systems/P2P networking depth: 30%-50% → 13.5k-118k

- Security engineering mindset (memory safety, adversarial thinking): 20%-30% → 2.7k-35k
- Economics/game-theory literacy: 30%-50%, resulting in 0.8k-17.5k
- Open-source governance (review culture, consensus norms): 10%-30%, resulting in 80-5k
- Communication/reliability under public scrutiny: 30%-50% → 24-2.5k
- Availability/alignment to work on Bitcoin: 10%-30% → ~2-750

Interpretation:

- Even with generous ranges, the intersection yields on the order of dozens to a few hundred globally available individuals with the full stack to work reliably on Bitcoin's most sensitive layers.
- Public data corroborates scarcity at the tip: Bitcoin Core has hundreds of credited contributors but only a small, rotating single-digit maintainer set. This human bottleneck contrasts with the cryptographic abundance at the consensus layer.

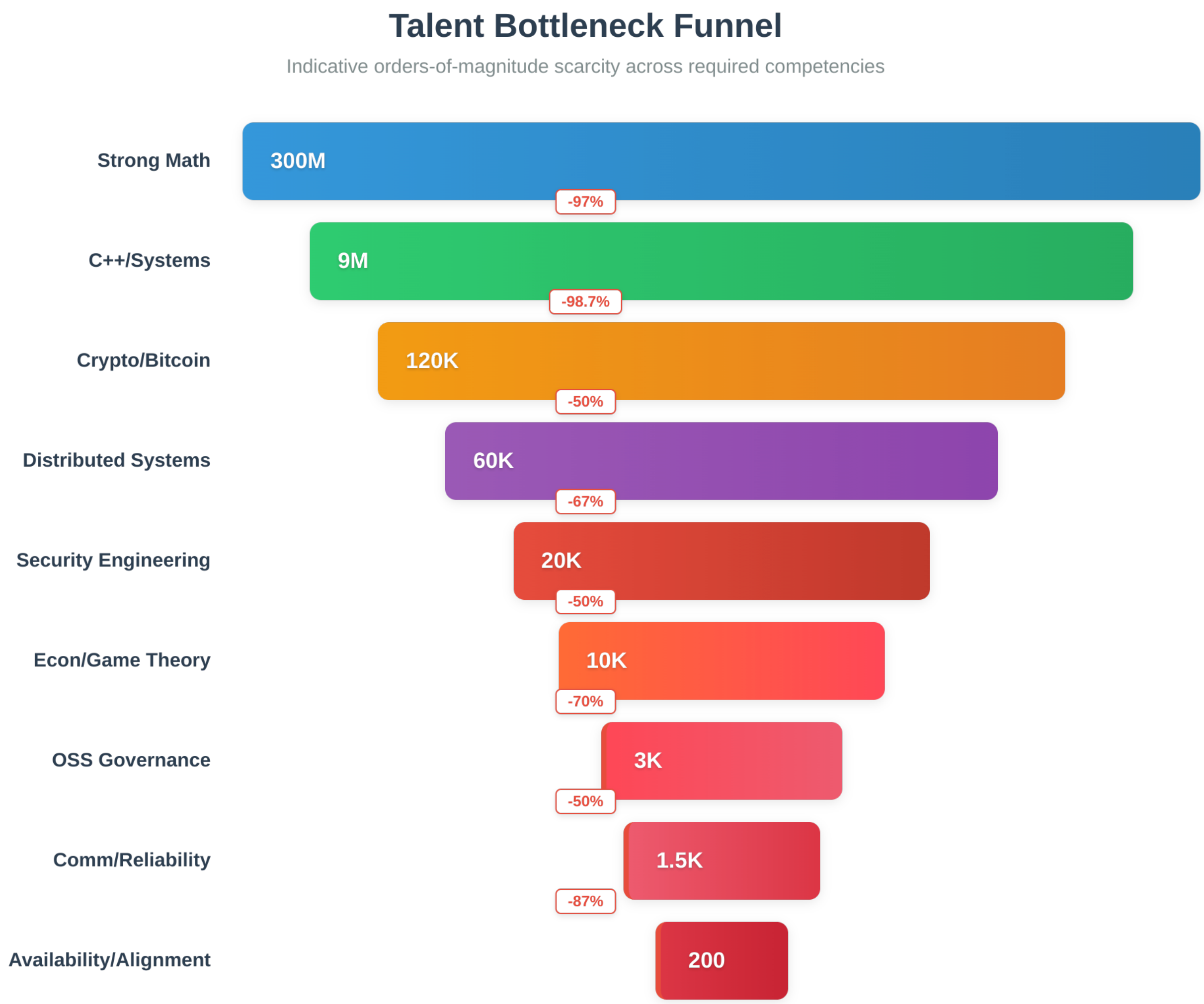

Figure: Orders of magnitude funnel showing talent scarcity across required domains.

Citations (illustrative anchors): - UN DESA, World Population Prospects (2022) - SlashData, Global Developer Population Trends - Annual developer surveys (Stack Overflow) for C++ usage - OECD PIAAC, adult skills numeracy distributions - Bitcoin Core repository statistics (GitHub)

2. Problem Statement

Technical Reality

Bitcoin's consensus rules are embedded in hundreds of thousands of lines of reference-client C++ without a canonical mathematical specification the ecosystem standardizes on. The dominant reference implementation carries enormous practical weight over how the network evolves. Safe independent implementations and systematic verification remain exceedingly difficult under reference-client-as-spec; Commons publishes layered specifications to address that gap.

Governance Reality

Bitcoin's development governance relies largely on informal social coordination. At network scale, consequences for violations are rarely systematic, dispute resolution is ad hoc, and authority is difficult to audit. The system is vulnerable to capture through relationships rather than rules. Network analysis reveals structural misalignments between technical development and social infrastructure, creating coordination gaps that prevent effective governance (Hough, 2025). These patterns reflect the paradox of embeddedness in network structures, where relationships can inhibit coordination and reinforce existing power dynamics (Uzzi, 1997). Funding may not flow to projects with strong grassroots activity, and "rich-get-richer" dynamics reinforce existing patterns rather than enabling competition. Commons addresses these gaps through cryptographic enforcement of multisig thresholds, mandatory review periods, and transparent accountability mechanisms.

Historical Context

Early leadership transitions and public governance disputes showed that developer coordination can fail without institutional enforcement. Andresen (2014) and Hearn (2016) raised governance concerns from inside development; academic researchers documented the structural gap but offered no implementable remedy (De Filippi & Loveluck, 2016).

Scale Considerations

Bitcoin's growth from early stages to multi-trillion dollar scale requires institutional reform. The next crisis, whether regulatory capture, internal conflict, or novel attack classes, will not wait for the community to develop governance solutions reactively.

3. Theoretical Framework: The Triple Foundation

Bitcoin Commons synthesizes three distinct theoretical frameworks, each addressing weaknesses in the others to create governance architecture stronger than any single approach alone.

Framework 1: Elinor Ostrom - Commons Governance

Elinor Ostrom won the 2009 Nobel Prize in Economics for proving that shared resources don't inevitably collapse into chaos or capture (Ostrom, 1990). Her research documented principles for governing commons without central authority across centuries of real-world examples.

Ostrom's (1990) Seven Principles.

1. **Clear boundaries on who decides what.** Defined decision-making authority.
2. **Consequences for violations.** Systematic enforcement mechanisms.
3. **Local dispute resolution.** Formal conflict resolution processes.
4. **Protection from external interference.** Resistance to outside pressure.

5. **Collective choice arrangements.** Meaningful participation in rule-making.
6. **Graduated sanctions.** Proportional consequences for violations.
7. **Monitoring and accountability.** Transparent oversight mechanisms.

What This Provides. Proven institutional design for shared resources, evidence decentralized governance works, and coordination without hierarchy.

Framework 2: F.A. Hayek - Spontaneous Order

Friedrich Hayek's Austrian economics provides the competitive discovery mechanism that enables governance evolution rather than rigid design.

Hayek's Core Insights. - **Dispersed Knowledge Problem.** No central planner can know what's needed because knowledge is distributed across many actors. - **Competition as Discovery.** Competition reveals information that couldn't be known in advance. - **Spontaneous Order.** Best institutions emerge through evolution, not top-down design. - **Markets Need Infrastructure.** Competition requires actual alternatives to compete.

What This Provides. Justification for avoiding central planning, competitive governance discovery, and institutions that evolve through market signals.

Framework 3: Bitcoin - Cryptographic Enforcement

Bitcoin's innovation provides the enforcement tools that make decentralized governance work at scale without trusted parties.

Bitcoin's Core Principles.

- **Don't Trust, Verify.** Cryptographic enforcement replaces social trust.
- **Permissionless Innovation.** Anyone can build without asking permission.
- **Exit Rights.** Fork option provides ultimate check on power.
- **Decentralized Control.** No single point of authority.

What This Provides. Tools for enforcing rules without trust, proof decentralized systems work at scale, and a model for implementing Hayek's principles digitally.

The Triple Synthesis

The three frameworks address each other's weaknesses:

Ostrom's Challenge. Commons governance historically relied on social pressure, vulnerable to capture at scale. **Bitcoin's Solution.** Cryptographic enforcement replaces social pressure with mathematical proof.

Hayek's Challenge. Competition discovers optimal solutions but requires actual alternatives to compete. **Ostrom's Solution.** Provides institutional framework for multiple governance models to coexist.

Bitcoin's Challenge. Solved technical consensus but not social governance. **Hayek + Ostrom Solution.** Competitive discovery of governance models using proven institutional principles.

The Result. Governance that is proven (Ostrom's research), evolving (Hayek's competition), and enforceable (Bitcoin's cryptography).

Cryptographic Polycentrism: The Governance Model

Bitcoin Commons implements **Cryptographic Polycentrism**, a governance model that creates multiple centers of power with clear boundaries, enforced through cryptography rather than social pressure. This approach applies Ostrom's proven principles using Bitcoin's cryptographic enforcement tools, enabling competitive discovery through Hayekian market mechanisms while maintaining Ostrom's institutional structure.

Cryptographic Polycentrism differs from traditional polycentrism (multiple decision centers) by using cryptographic enforcement to make boundaries mathematically verifiable rather than socially negotiated. Where traditional governance relies on trust and reputation, Cryptographic Polycentrism uses multisig thresholds, mandatory review periods, and three-layer verification providing real-time transparency and immutable proof to create enforceable boundaries that cannot be bypassed even by repository administrators.

This model enables multiple implementations to compete (Hayek's discovery mechanism) while maintaining clear institutional boundaries (Ostrom's principles) through cryptographic proof (Bitcoin's innovation). The result is governance that is both competitive and accountable, proven and evolving, decentralized and enforceable.

Informal governance can work at smaller scale and lower complexity; at multi-trillion dollar scale, social enforcement alone struggles to keep pace with coordination demands and accountability expectations. Commons encodes Ostrom's principles through cryptographic enforcement. The mapping below describes how each principle is implemented in this architecture.

Mapping The Principles to Implementation

The modular architecture implements Ostrom's seven principles through cryptographic enforcement rather than social pressure. The chart below shows how these principles integrate with principles from Hayek, Bitcoin, and Cypherpunk frameworks:

Principle	Source	Technical Implementation
Clear Boundaries Who has authority, what belongs in base vs extensions, entry/exit rules for maintainers	Ostrom	**Repository hierarchy with signature thresholds:** Constitutional (6-of-7), Implementation (4-of-5), Application (3-of-5), Extension (2-of-3). Each repository has defined scope and authority boundaries enforced by multisig. `governance-app/src/crypto/multisig.rs`
Proportional Benefits Costs and benefits align with participation level, no free-riding by maintainers	Ostrom	**Economic node network and module marketplace:** Economic nodes receive proportional voting weight based on stake. Module developers earn marketplace revenue based on adoption. Merge mining rewards align with contribution to network security. `governance-app/src/economic_nodes/registry.rs`
Collective Choice Participants design governance rules, not imposed from above	Ostrom	**Module system enables user sovereignty:** Users choose which modules to run, creating bottom-up governance through configuration rather than top-down feature decisions. Module marketplace enables competitive discovery of solutions. `reference-node/ (modular architecture)`
Monitoring Transparent oversight, audit trails, verifiable governance actions	Ostrom	**Three-layer verification architecture:** GitHub enforcement (real-time merge blocking), Nostr transparency (hourly signed status), OpenTimestamps anchoring (monthly immutable proof). All governance actions are cryptographically signed and verifiable. `governance-app/src/verification/`
Graduated Sanctions Proportional consequences for violations, not just fork-or-nothing	Ostrom	**Tiered security controls and emergency response:** P0 controls block production, P1 controls require audit, P2 controls warn. Emergency powers have automatic expiration. Module quality gates provide graduated enforcement (warnings → rejection → removal). `governance-app/src/enforcement/security_controls.rs`
Conflict Resolution Local mechanisms for disputes, formal processes not ad-hoc	Ostrom	**OpenTimestamps provides court-admissible evidence:** Monthly canonical registry anchored to Bitcoin blockchain creates immutable proof of governance state. Enables formal dispute resolution with cryptographic evidence rather than ad-hoc social negotiation. `governance-app/src/verification/opentimestamps.rs`
Minimal Recognition Autonomy from external interference, protection from regulatory capture	Ostrom	**Multi-jurisdictional maintainer distribution:** 5-7 maintainers across multiple jurisdictions. Cryptographic enforcement means even repository admins cannot bypass multisig requirements. No single point of regulatory pressure. `governance-app/src/crypto/multisig.rs`
Spontaneous Order Emergent coordination without central planning, bottom-up innovation	Hayek	**Module marketplace enables emergent solutions:** No central committee decides features. Module developers build solutions based on market demand. Users vote with configuration choices. Innovation emerges from competition, not planning. `Module system (planned architecture)`
Knowledge Problem Distributed knowledge utilization, no central bottleneck for decisions	Hayek	**Modular architecture distributes decision-making:** Module system allows distributed knowledge application. Different modules solve different problems without requiring central coordination. Economic nodes contribute local knowledge through voting signals. `reference-node/ (modular architecture)`
Price Discovery Market signals guide resource allocation, module marketplace enables choice	Hayek	**Module marketplace pricing and adoption metrics:** Module sales, adoption rates, and user ratings provide market signals. Popular modules get more resources and development. Unpopular modules exit. Price signals guide developer effort allocation. `Module system (planned architecture)`
Market Process Competitive discovery and adaptation, multiple implementations compete	Hayek	**Orange Paper enables implementation diversity:** Mathematical specification allows multiple implementations to compete while maintaining consensus compatibility. Module system enables experimentation without consensus risk. Multiple implementations can coexist and compete. `the-orange-paper/`
Permissionless No gatekeeping for participation, anyone can fork and contribute	Bitcoin	**Open source with fork-ready architecture:** All repositories public. Modular design makes forking easier. Module system allows contribution without touching core consensus. Anyone can create modules, fork implementations, or start new implementations from Orange Paper. `All repositories public, fork-ready`
Trust Minimization Cryptographic verification over trust, governance enforced in code	Bitcoin	**Multisig enforcement blocks unauthorized merges:** 3-of-5 maintainer multisig required for merges, enforced by GitHub App. Even repository admins cannot bypass. Signatures are cryptographically verified. Governance is enforced in code, not through social trust. `governance-app/src/enforcement/merge_block.rs`
Decentralization Distributed control and validation, no single point of failure	Bitcoin	**Multi-stakeholder maintainer set across jurisdictions:** 5-7 maintainers required for operations. Economic node network provides distributed validation. Module system prevents single implementation monopoly. Multiple implementations can validate same chain. `governance-app/src/validation/threshold.rs`
Censorship Resistance Resistance to external pressure, no single jurisdiction control	Bitcoin	**Jurisdictional distribution and cryptographic enforcement:** Maintainers across multiple jurisdictions. Cryptographic enforcement prevents single-jurisdiction shutdown. Self-funding through merge mining reduces external pressure. No single point of regulatory control. `governance-app/src/validation/threshold.rs`
Immutability Permanent historical record, governance decisions cryptographically anchored	Bitcoin	**OpenTimestamps anchors governance state to Bitcoin:** Monthly canonical registry hash anchored to Bitcoin blockchain via OpenTimestamps. Creates permanent, court-admissible proof of governance state. Independent of any single server or relay. `governance-app/src/verification/opentimestamps.rs`
Privacy by Default User privacy as fundamental right, governance doesn't require surveillance	Cypherpunk	**Governance operates without user surveillance:** Multisig verification doesn't require monitoring users. Module choices are local configuration, not tracked. OpenTimestamps proofs don't reveal private information. Privacy-preserving governance design. `governance-app/src/crypto/`
Technical Enforcement Code enforces rights and limits, multisig prevents governance bypass	Cypherpunk	**Cryptographic enforcement throughout:** Multisig blocks unauthorized merges at GitHub level. Security controls enforced in CI/CD. Signatures verified mathematically. Code enforces governance rules; humans cannot override without cryptographic proof. `governance-app/src/enforcement/merge_block.rs`
Anti-Surveillance Resistance to monitoring and control, decentralized governance infrastructure	Cypherpunk	**Distributed infrastructure and privacy-preserving design:** Nostr relays distributed across network. OpenTimestamps uses public Bitcoin blockchain (no surveillance risk). Self-hosted runners behind VPN. No central monitoring infrastructure required. `governance-app/src/verification/nostr.rs`

Figure: Integration of four key philosophies: Hayek (spontaneous order), Bitcoin (cryptographic enforcement), Cypherpunk (privacy through technology), and Ostrom (commons governance).

Principle 1. Clear Boundaries.

- **Layers.** Base (network consensus), Module (user choice).
- **Implementation.** Architecture enforces boundaries. Modules cannot modify consensus code paths.

Principle 2. Consequences for Violations.

- **Technical/Reputational.** Module quality standards, transparent adoption metrics, cryptographic enforcement.
- **Implementation.** Cryptographic enforcement makes consequences automatic, not social.

Principle 3. Local Dispute Resolution.

- **Architecture-based.** Competing modules resolve disputes. User choice determines winners. Module conflicts don't threaten consensus.
- **Implementation.** No central arbiter needed. Architecture provides resolution through user configuration.

Principle 4. Protection from External Interference.

- **Self-Funding/Multi-jurisdictional/Fork-Ready.** Module-based revenue, distributed keyholders, governance fork capability.
- **Implementation.** Cryptographic multisig ensures no single jurisdiction can compel action.

Keyholder Diversity Radar

Phase 3 estimates. Higher is better across axes: jurisdictions, org diversity, rotation cadence, independence, quorum.

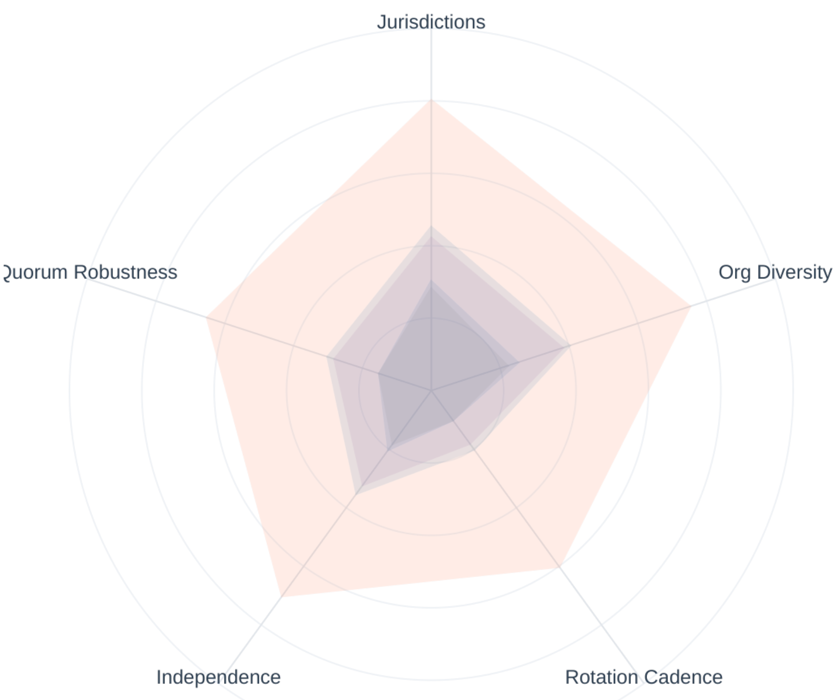

Figure: Keyholder diversity across jurisdictions and organizations prevents single-point coercion.

Principle 5. Collective Choice Arrangements.

- **User sovereignty via configuration.** Users compose stacks. Adoption metrics function as voting. Participation through choices, not committees.
- **Implementation.** Collective choice is expressed through **configuration**: declarative module composition (and node/SDK configuration) rather than a single mandatory GUI; operator-facing UX may include CLI, config files, RPC, and optional UI as tools mature.

Principle 6. Graduated Sanctions.

- **Proportional escalation.** Moderate fork to major deprecation to governance fork. Graduated signature requirements (2-of-3 to 6-of-7). Sanctions at module layer, no consensus changes required.
- **Implementation.** Multisig thresholds vary by change category (2-of-3 extension to 6-of-7 constitutional).

Principle 7. Monitoring and Accountability.

- **Cryptographic transparency.** All governance actions signed and verifiable. Module adoption, revenue flows, decision provenance auditable. Three-layer verification ensures complete audit trails.
- **Implementation.** Automated monitoring through cryptographic verification, not social trust.

Audit Trail Completeness Map

Phase 3 estimates. Cells indicate availability of verifiable artifacts across implementations.

	Bitcoin Core	Bitcoin Knots	btcd	Libbitcoin	Bitcoin Commons
Release Signatures	Complete	Complete	Partial	Partial	Complete
Deterministic Builds Proof	Partial	Partial	Partial	Partial	Complete
Review Logs (linked to PRs)	Partial	Partial	Partial	Partial	Complete
CI Verification Proofs	Partial	Partial	Partial	Partial	Complete
OpenTimestamps / Timestamps	Partial	Partial	Partial	Partial	Complete
Merkle Proofs (Commit Sets)	No	No	No	No	Complete

Figure: Audit-trail completeness across governance layers: all decisions evidenced and verifiable.

Commons implements all seven Ostrom principles through Cryptographic Polycentrism, encoding boundaries, consequences, and auditability in architecture rather than social pressure alone.

4. Technical Solution: The Orange Paper

Problem

Bitcoin's consensus rules lacked a canonical mathematical specification industry-wide (reference-client-as-spec remains the practical default). The consensus-critical denial-of-service vulnerability CVE-2018-17144 existed undetected for years: exactly the class of error formal verification and layered testing aim to prevent.

Solution

The Orange Paper provides a formal mathematical specification of Bitcoin's consensus protocol through AI-assisted extraction from Bitcoin Core's codebase. The specification includes:

- Mathematical foundations (set theory, cryptographic primitives, network protocols)
- State transition functions (block validation, transaction validation, consensus rules)
- Economic model (mining rewards, fee calculations, difficulty adjustment)
- Security properties (Byzantine fault tolerance, Sybil resistance, double-spending prevention)

Benefits

- **Safe alternative implementations**: Independent implementations can verify against mathematical specification

- **Formal verification**: Consensus-critical code is checked against the Orange Paper through **BLVM Specification Lock**, property tests, and CI; see Formal Verification
- **Reduced consensus bugs**: Systematic analysis eliminates entire classes of errors

AI-Assisted Extraction Methodology

The Orange Paper uses AI-assisted extraction from Bitcoin Core's codebase to formalize consensus rules. This approach: - Analyzes Bitcoin Core's codebase to identify consensus-critical code paths - Extracts mathematical relationships from implementation details - Creates formal specifications that are independent of specific code structure - Enables verification that specification matches actual network behavior

Proof Maintenance and Specification Quality

The formal verification process includes ongoing maintenance to ensure specification accuracy:

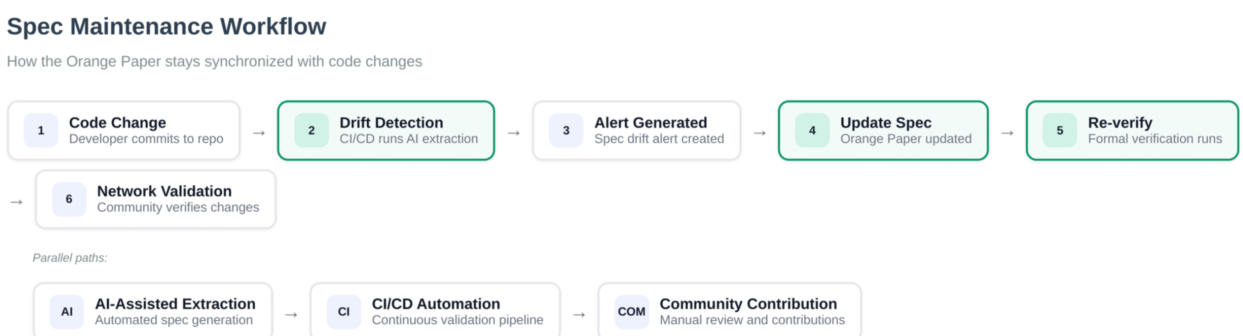

Figure: Spec maintenance workflow: specification synchronized with implementation through automated testing and formal verification.

Commons publishes a **layered specification**: the **Orange Paper** (formal math), **CONSENSUS_SPEC** (numbered RFC 2119 rules mapped to code), and **PROTOCOL** (extended constants and formulas). Entry points are at thebitcoincommons.org/spec.html. **blvm-spec-lock** ties `blvm-consensus` to these artifacts; drift and coverage tooling tracks alignment between spec wording and lockable contracts. Details: Orange Paper, Formal Verification.

Empirical consensus alignment (differential testing and fuzzing)

Mathematical specification and spec-lock contracts bound *what the rules ought to be*. **Differential testing** and **fuzzing** ask whether the running implementation agrees with the dominant reference implementation on real chain history and on adversarial inputs. Methodology, current depth, and CI status are documented in technical documentation. Prefer that over this document for live coverage figures.

Together these layers complement formal methods: **specs and contracts** for specification fidelity; **replay and fuzzing** for empirical alignment with observed network behavior.

BLVM Architecture

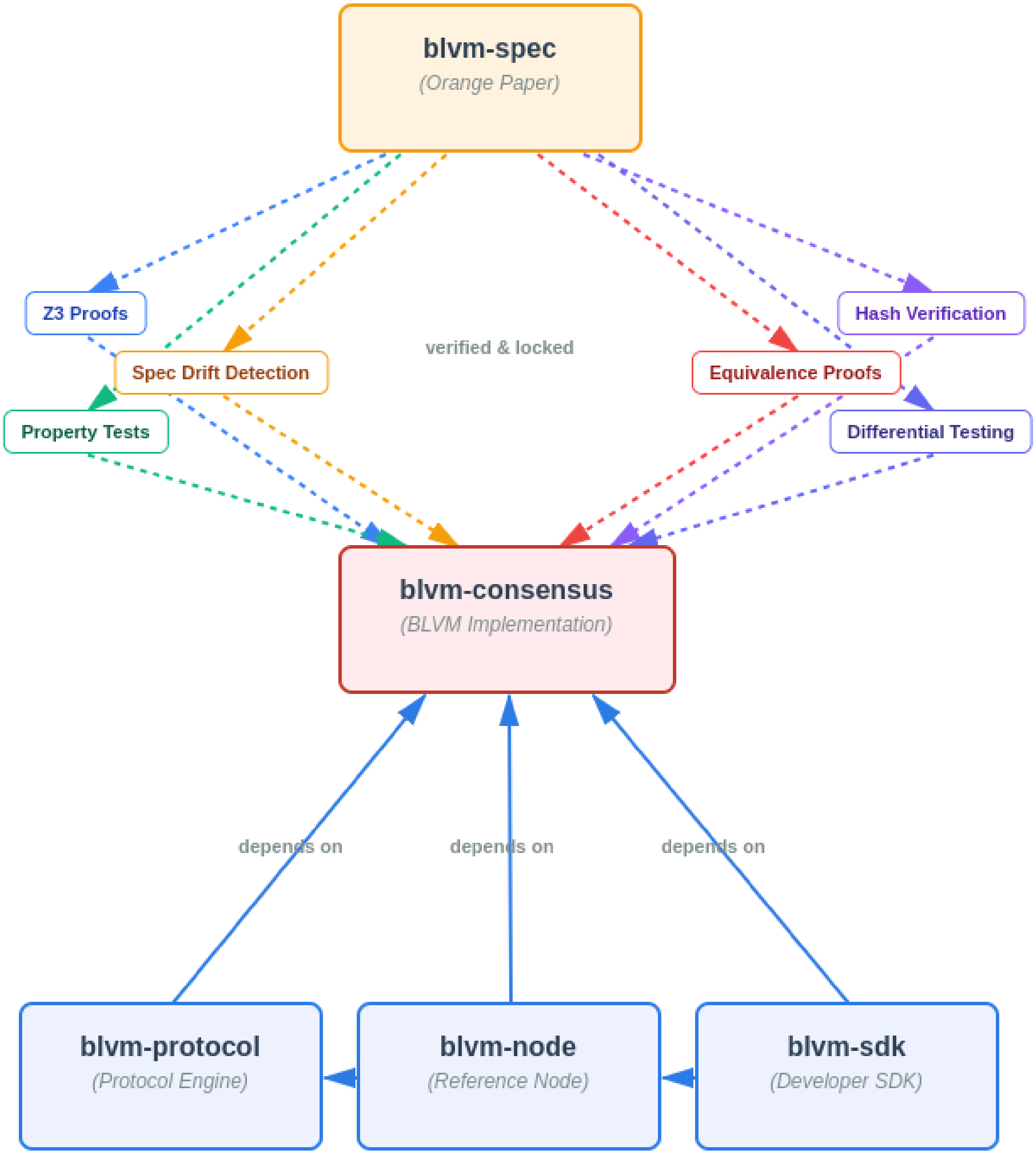

Figure: BLVM architecture: blvm-spec (Orange Paper) as foundation, blvm-consensus as verified core, and dependent components (blvm-protocol, blvm-node, blvm-sdk) building on the consensus layer.

BLVM (**Bitcoin Low-Level Virtual Machine**) applies a **compiler-like** pipeline to Bitcoin implementations. The Orange Paper acts as an intermediate representation (IR), analogous in role to IRs in compiler toolchains. The Orange Paper serves as that IR, enabling reusable optimizations and multiple implementations. Stack layout and verification methodology: System Overview, Consensus Overview.

Single Source of Truth. All consensus logic resides in `blvm-consensus`. Upper tiers (`blvm-protocol`, `blvm-node`) delegate validation calls with no duplicate implementations. Path dependencies ensure changes propagate immediately through Rust's type system.

Because every consensus decision flows through `blvm-consensus`, verification and tests at that layer bound behavior for the whole stack: upper tiers delegate validation and cannot bypass that layer.

5. Architectural Solution: Modular Governance

Two innovations work together: **BLVM** provides the mathematical foundation and compiler-like architecture (Orange Paper as IR, formal verification passes); **Commons** provides the governance framework (coordination without civil war). The modular architecture is where both innovations meet. BLVM ensures correctness through architectural enforcement; Commons ensures coordination through Cryptographic Polycentrism.

Commons distributes authority through cryptographic multisig requirements and mandatory review periods, preventing single points of control at the merge gate. This Cryptographic Polycentrism model creates multiple decision centers (repository layers, action tiers, emergency tiers) with mathematically enforced boundaries.

Two-Layer Architecture

The modular architecture consists of two layers that transform governance conflicts from political battles into architectural choices:

Layer 1: Mandatory Consensus (Base Node)

- Bitcoin's consensus rules, unchangeable without network agreement
- Cryptographically enforced, defines what "Bitcoin" means
- Examples: block validation, transaction validation, fork choice rules

Layer 2: Optional Modules (Extension System)

- User-controlled features that can be enabled or disabled
- Communities can fork/modify/compete, user choice determines winners
- Examples: Lightning Network, Taproot Assets, privacy enhancements, Stratum/mining-pool interfaces, optional merge mining where enabled

Revenue Model

- Self-sustaining development through optional modules
- Three complementary channels; no single stream is required: **optional modules** (operational today), **module registry and distribution** (not live yet; discovery and fees when deployed), **governance announcement zaps** (Lightning), and **optional merge mining** for pools that choose secondary-chain revenue
- Reduces reliance on grants, donations, or external funding
- Aligns incentives with usage and adoption, reducing capture risk

Module Isolation

Optional features load as **modules**: separate processes with strict boundaries, not compile-time features baked into the base node:

Process Isolation Mechanisms: - Each module runs in its own process with isolated memory - Modules communicate through a documented **ModuleAPI** (subprocess IPC) - Base node validates all blocks using Orange Paper specification regardless of enabled modules - Module state is separate from consensus state (UTXO set)

API Boundaries: - Modules interact with the base layer only through documented interfaces - No direct access to consensus functions or core data structures - Module failures cannot propagate to the base node

What modules CANNOT do: Modify consensus rules, alter block validation, cause network splits

What modules CAN do: Provide optional services (relay, mining interfaces, Lightning, ZMQ, etc.) without affecting base-node stability

Containment Strategy

The modular architecture satisfies both camps simultaneously:

- **"Don't Change Bitcoin" Camp**: Gets pure Bitcoin base layer with no modifications
- **"Make Bitcoin Useful" Camp**: Gets optional features through modules
- **Miners and pools**: Stratum V2 connectivity to the ecosystem; optional merge mining only where operators enable it, not the primary sustainability bet

The Module System IS The Governance System: Instead of governing through committees deciding features, Commons governs through architecture enabling choice. The module system isn't just technical: it's the governance mechanism itself, implementing Ostrom's collective choice arrangements through user configuration, Hayek's competitive discovery through module competition, and Bitcoin's permissionless innovation through fork-ability.

Architecture Diagrams

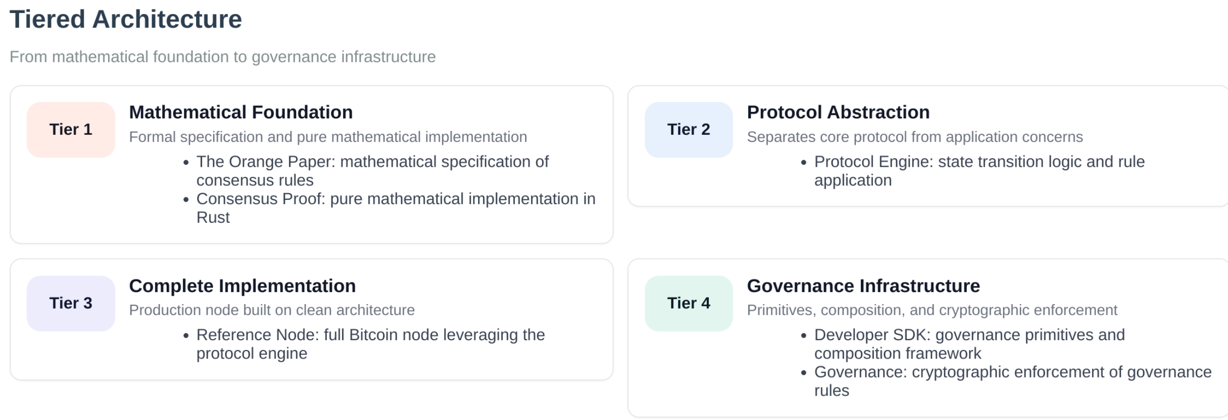

Figure: Tiered architecture: Tier 1 = Orange Paper + Consensus Proof (mathematical foundation); Tier 2 = Protocol Engine (protocol abstraction); Tier 3 = BLVM Node (complete implementation); Tier 4 = Developer SDK + Governance (developer toolkit + governance enforcement).

Figure: End-to-end data flow through BLVM Node, Consensus Proof, Protocol Engine, modules, and governance. Each tier depends only on layers below; modules cannot affect consensus.

Fragmentation Analysis:

Governance forks preserve the consensus layer while allowing governance changes. Users can fork governance rules while keeping the same Bitcoin consensus. This is the ultimate accountability mechanism. **Alternate full-node implementations** (for example Bitcoin Knots) illustrate that materially different clients can coexist without splitting the network when validation rules remain aligned (governance and packaging can vary while consensus stays unified). Reported adoption percentages vary by measurement window and methodology; the structural point is **coexistence without consensus fragmentation**, not a specific market-share figure.

Forking Process: Governance rulesets can be exported as versioned, signed packages containing action tiers, maintainers, cryptographic proofs, and semantic versioning. This enables users to fork governance while maintaining compatibility and verification.

Optional **modules** are operational today (loadable processes via ModuleAPI). A **module registry** for permissionless listing and automated validation is part of the distribution model but **not live yet**.

6. Cryptographic Governance Enforcement

Scope: Merge gates, production keyholders, and full three-layer transparency are **designed**; **production activation** follows prerequisites documented in the governance repository. This section describes intent, not guaranteed live operation.

Commons implements Cryptographic Polycentrism through three complementary verification layers (distinct from repository layers) that ensure both real-time transparency and immutable historical proof. These verification layers create multiple independent centers of accountability, each enforcing governance boundaries through cryptographic proof:

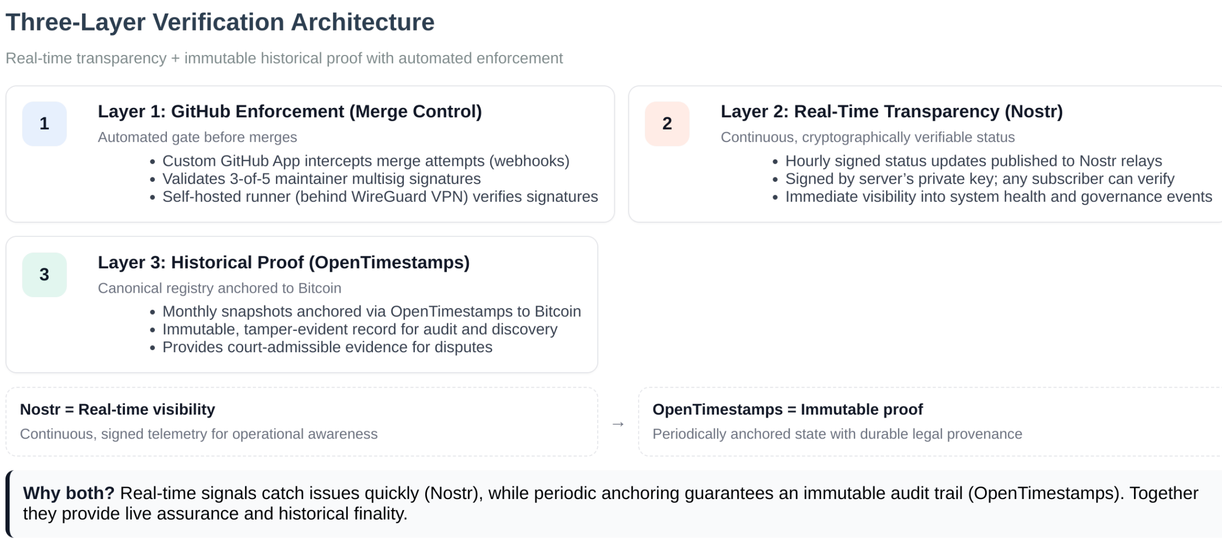

Figure: Three-layer verification approach: GitHub, Nostr, and OpenTimestamps.

Layer 1: GitHub Enforcement (Merge Control). *Designed to*, when production governance is activated, enforce merge policy through the following:

- Custom GitHub App validates multisig requirements (varies by layer: 2-of-3 to 6-of-7)
- Self-hosted runner behind WireGuard VPN validates signatures using secp256k1
- Even repository admins cannot bypass cryptographic requirements
- Signature validation happens before merge approval
- Requires explicit cryptographic approval from multiple maintainers before any merge
- **No-self-merge:** a contributor cannot cryptographically approve their own pull request; the Governance App rejects self-signed merges

Layer 2: Real-Time Transparency (Nostr). *Designed to* publish signed status updates (binary hash, config hash, recent merges, health metrics) to Nostr relays so subscribers can verify server integrity. Cadence and alerting are deployment parameters, not guarantees about any particular live deployment.

Layer 3: Immutable Proof (OpenTimestamps). *Designed to* anchor canonical registry snapshots and critical events (key rotations, deployments) to the Bitcoin blockchain for dispute resolution and independent verification. Provides court-admissible evidence where applicable.

Cross-Layer Verification: Three independent layers verify governance actions and each other. Risk at one layer does not compromise the others. This defense-in-depth approach ensures governance integrity even if one verification method is compromised.

Emergency Response

Commons implements a three-tier emergency system for critical security and network issues. **Emergency tier activation** is separate from **action tier** classification (see below): emergency keyholders declare the class; merge rules and review periods adjust until automatic expiration.

Activation (all classes): 5-of-7 emergency keyholders. **Merge policy while active** varies by class: network-threatening incidents use the fastest path (zero review, lower maintainer merge threshold); less urgent classes use longer review and stricter merge thresholds. Authoritative parameters: governance repository (`emergency-tiers.yml`).

Critical Emergency (Network-threatening vulnerabilities): - Merge signatures while active: 4-of-7 - Review period: 0 days (immediate response) - Maximum duration: 7 days (no extensions) - Examples: Consensus-critical denial-of-service bugs, consensus fork risks, P2P network DoS vulnerabilities, remote code execution

Urgent Security Issue (Serious security issues): - Merge signatures while active: 5-of-7 - Review period: 7 days - Maximum duration: 30 days (1 extension allowed) - Examples: Memory corruption vulnerabilities, privacy leaks, crash exploits, privilege escalation

Elevated Priority (Important but not critical): - Merge signatures while active: 6-of-7 - Review period: 30 days - Maximum duration: 90 days (2 extensions allowed) - Examples: Competitive response needs, important bug fixes, performance degradation, ecosystem compatibility issues

The system scales **response speed and review strictness** to severity, not a single monotonic rule that stricter always means worse. Network-threatening issues get the fastest merge path; important-but-non-critical work uses longer review and higher merge bars. All emergency actions require post-mortem documentation, automatic expiration, and review after expiration.

Security Architecture: Push-Only Design

Security Architecture Details:

- **No HTTP Endpoints**: Governance servers have no incoming HTTP endpoints (minimal exposure surface)
- **VPN Isolation**: Servers communicate outbound only through WireGuard VPN

- **Self-Hosted Runner**: GitHub runner behind WireGuard VPN for signature validation
- **Data Flow**: Server to GitHub (push) to Nostr (publish) to Bitcoin (anchor)
- **Public Read Access**: GitHub repo, Nostr relays, Bitcoin blockchain (read-only for public)

Attack Path Protection:

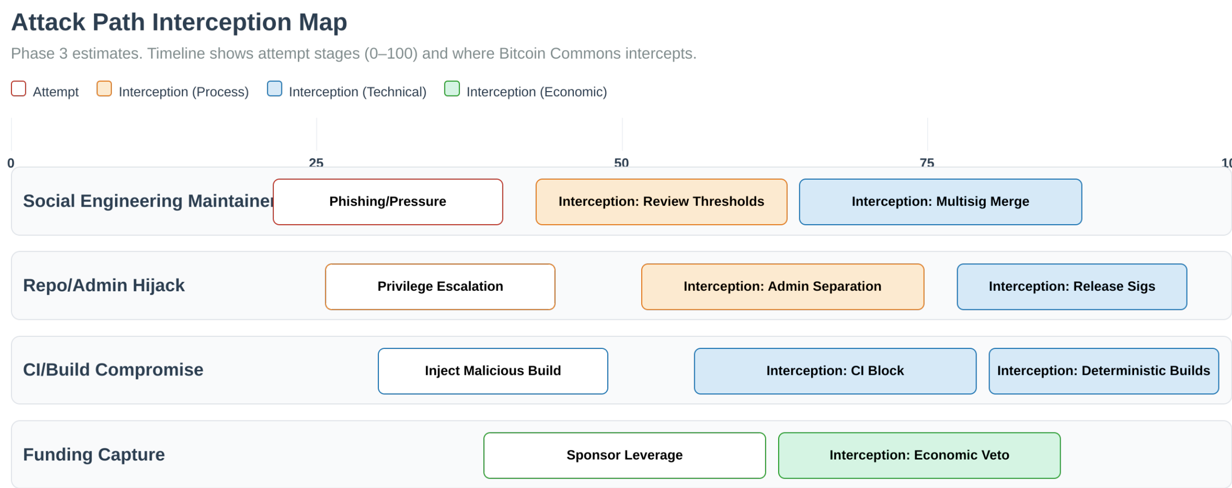

Figure: Risk interception points across three independent verification layers.

Multisig Threshold Details

Authoritative configuration. Signature thresholds, repository **layer** definitions, **action tier** names, review periods, and **emergency** parameters described in this section are **illustrative** of the governance *design*. **Security-class overrides** (for example consensus-critical or cryptographic paths) may impose stricter requirements than the tables below; see `action-tiers.yml` in the governance repository. **Authoritative numeric requirements and rule text** live in that repository (YAML and related artifacts) and may change as policy evolves. This PDF is not the source of truth for merge policy; when in doubt, consult the repo.

Commons implements a **5-tier constitutional governance system** that combines two dimensions: **repository layers** (where the change is made) and **action tiers** (what type of change is being made). When both apply, the **most restrictive requirement wins** (highest signature threshold and longest review period).

Repository Layers (Where): - **Layer 1-2 (Constitutional)**: Orange Paper and Consensus Proof - 6-of-7 signatures, 180-day review (365 days for consensus changes) - **Layer 3 (Implementation)**: Protocol Engine - 4-of-5 signatures, 90-day review - **Layer 4 (Application)**: BLVM Node - 3-of-5 signatures, 60-day review - **Layer 5 (Extension)**: SDK and modules - 2-of-3 signatures, 14-day review

Action Tiers (What): - **Tier 1 (Routine Maintenance)**: Bug fixes, documentation, optimizations - 3-of-5 signatures, 7-day review - **Tier 2 (Feature Changes)**: New RPC methods, P2P changes, wallet features - 4-of-5 signatures, 30-day review - **Tier 3 (Consensus-Adjacent)**: Changes affecting consensus validation code - 5-of-5 signatures, 90-day review - **Tier 4 (Emergency Actions)**: Critical security patches, network-threatening bugs - 4-of-5 signatures, 0-day review (with post-mortem required) - **Tier 5 (Governance Changes)**: Changes to governance rules themselves - 5-of-5 signatures, 180-day review

Emergency PRs vs emergency activation. *Action Tier 4* classifies pull requests for critical security work under normal maintainer multisig. *Emergency tier activation* (above) is a separate declaration by emergency keyholders that switches merge rules until expiration. Both can apply together; see `emergency-tiers.yml` and `action-tiers.yml` in the governance repository.

Most Restrictive Wins Rule: When both layer and tier apply, the system takes the highest signature requirement and longest review period. For example: - A routine bug fix (Tier 1) in the Protocol Engine (Layer 3) requires 4-of-5 signatures and 90-day review (Layer 3 wins) - A new feature (Tier 2) in the SDK (Layer 5) requires 4-of-5 signatures and 30-day review (Tier 2 wins) - A consensus change (Tier 3) in Orange Paper (Layer 1) requires 6-of-7 signatures and 180-day review (Layer 1 wins)

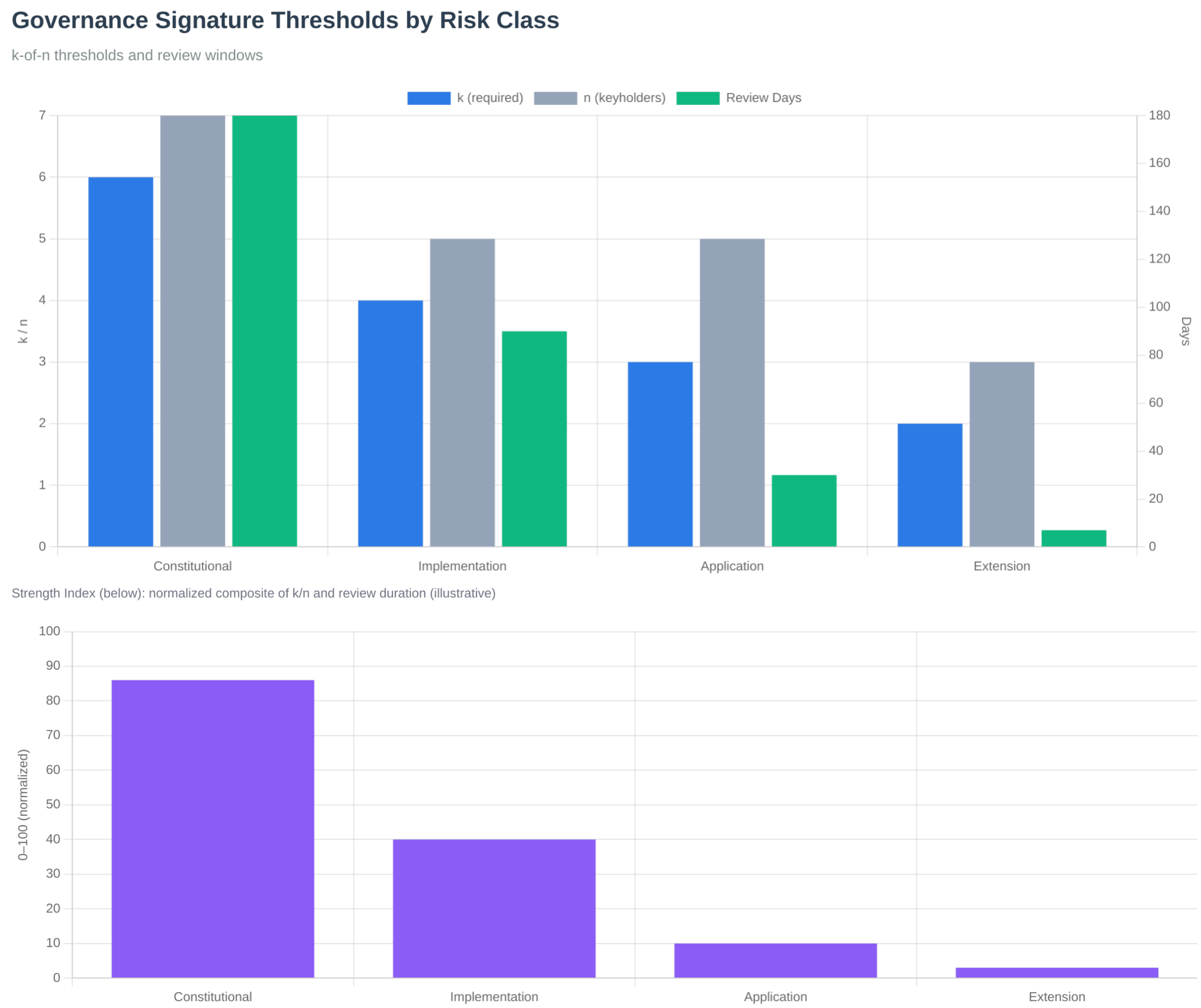

Figure: Governance signature thresholds by change category (constitutional, implementation, application, extension).

All signatures verified using secp256k1 (same curve as Bitcoin). GitHub App validates signatures before allowing merges. Even repository admins cannot bypass cryptographic requirements.

Commons requires distributed approval: even the highest-risk changes (constitutional) require 6-of-7 signatures, ensuring no single maintainer or small group can control critical decisions. Mandatory review periods prevent rushed merges; the dual-dimensional system (layers + tiers) ensures even routine changes in critical repositories meet higher approval thresholds.

Contributor standing (proposed)

Beyond repository-layer multisig, a **contributor-standing ladder** complements merge policy: ledger-derived standing, crate-scoped participation, **no-self-merge** enforcement at the merge gate, and an Inquiry mechanism as a pause, not a veto, for contested changes. Described in a separate specification to be published when cryptographic governance activates. **Not enforced today.** The live governance repository defines maintainer multisig and tiers only.

7. Economic Sustainability

The Funding Gap

Only $8.4 million from 13 organizations supported Bitcoin Core development in 2023, while the network reached a $2 trillion market cap (Hough, 2025). That is roughly **0.00042%** of market cap, illustrating systemic under-funding relative to network value.

*The funding and market-cap figures are **snapshots** from the cited work (2023 funding year; market cap at the time of that analysis). They illustrate scale mismatch; they are not live dashboards.*

Self-Sustaining Revenue Models

Bitcoin Commons is built to fund operations through optional modules, which are operational today, reducing reliance on grants and donations. Sustainability is **multi-channel**: module revenue, registry-based distribution (when the registry is live), Lightning zaps, and optional mining-side revenue; none is assumed to carry the project alone.

Module registry and distribution: Planned permissionless discovery, quality signals, and developer alignment with project revenue; **not live yet**.

Governance announcement zaps: Lightning zaps attached to governance announcements fund governance operations directly and tie community support to transparent announcements.

Optional merge mining: For pools that enable it, coordinates merge mining for secondary chains as **supplemental** revenue, not a prerequisite for the economic model.

These self-sustaining models align incentives with usage and adoption, making the system less vulnerable to capture than grant-dependent funding models. Commons positions as **infrastructure for multiple implementations**, not a reference-client replacement: success is measured by ecosystem health and adoption of the model, not market share.

8. Failure Modes & Mitigations

Governance Capture

Risk: Keyholder collusion or compromise **Mitigation**: Multi-jurisdictional keyholders, transparent operation, fork-ready design. Informal governance at scale is easier to capture through relationships than architecture with enforced boundaries.

Regulatory Pressure

Risk: Authorities pressure keyholders to implement backdoors **Mitigation**: Distributed keyholders across jurisdictions (no single jurisdiction can compel 3-of-5 threshold), visible capture attempts, modular containment

Technical Risks

Risk: Module consensus bugs, complexity explosion **Mitigation**: Module isolation, formal verification, security audits

Social Risks

Risk: Community rejection, fork wars, reputation attacks **Mitigation**: Focus on substance, build alternatives, let market decide; not asking permission, let code speak, coalition provides proof

Ultimate Protection

Governance forks provide the ultimate accountability mechanism.

9. Conclusion

Bitcoin's governance vacuum represents its greatest vulnerability at multi-trillion dollar scale. The technical architecture is bulletproof, but the social architecture runs on gentleman's agreements. BLVM and Commons provide concrete, implementable solutions: BLVM ensures mathematical rigor; Commons applies Ostrom's principles, Hayek's competitive discovery, and Bitcoin's cryptographic enforcement to governance.

This isn't speculation. It's applying battle-tested principles from economics, social science, and cryptography to governance. Each framework addresses weaknesses in the others: cryptography makes Ostrom enforceable at scale, infrastructure enables Hayek's competition, and modularity plus fork-ability creates competitive discovery.

The foundation and architecture exist in public repositories. The project's future depends on community participation.

The choice: decentralize the builders, or watch them become kings.

References

Academic Sources

- Ostrom, E. (1990). *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press.
- Hayek, F. A. (1945). The Use of Knowledge in Society. *American Economic Review*, 35(4), 519-530.
- De Filippi, P., & Loveluck, B. (2016). The Invisible Politics of Bitcoin: Governance Crisis of a Decentralized Infrastructure. *Internet Policy Review*, 5(3).
- Hough, E. (2025). "Funding the Peer-to-Peer Ethos: A Network Analysis Proposal of Bitcoin's Technical & Social Collectives." Unpublished research proposal, INFO 4360: Communication Networks and Social Capital, Cornell University.
- Uzzi, B. (1997). Social Structure and Competition in Interfirm Networks: The Paradox of Embeddedness. *Administrative Science Quarterly*, 42(1), 35-67. https://doi.org/10.2307/2393808

Historical Sources

- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf
- Andresen, G. (2014). Bitcoin: The Future of Money? Princeton University, March 27, 2014.
- Hearn, M. (2016). The Resolution of the Bitcoin Experiment. *Medium*, January 15, 2016.

Technical Sources

- CVE-2018-17144 (2018). Bitcoin Core Denial of Service Vulnerability. *CVE Details*. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-17144
- Bitcoin Core Statistics. Public GitHub repository data.
- Bitcoin Optech Topics: High quality technical primers and references
  - Merged mining: https://bitcoinops.org/en/topics/merged-mining/
  - Stratum v2: https://bitcoinops.org/en/topics/stratum-v2/
- BOLT (Lightning) Specifications: https://github.com/lightning/bolts

Repository Links

- https://github.com/BTCDecoded/blvm-spec
- https://github.com/BTCDecoded/blvm-spec-lock (Orange Paper-to-Rust contract verification: `cargo spec-lock verify`)
- https://github.com/BTCDecoded/blvm-protocol
- https://github.com/BTCDecoded/blvm-consensus
- https://github.com/BTCDecoded/blvm-node
- https://github.com/BTCDecoded/blvm (packaging and release surface)

- https://github.com/BTCDecoded/blvm-sdk
- https://github.com/BTCDecoded/governance (governance configuration)
- https://github.com/BTCDecoded/blvm-commons (GitHub App that enforces governance)